Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Önceki dersin tekrarı
  - Veri sıkıştırma, yerden (fiziksel alan) ve zamandan (iletim süresi) kazanmak için yapılır. Çoğu zaman tek bir yöntemle yapılmaz; önüne ya da arkasına ön işlemler eklenir. Görüntü sıkıştırmada önce nicemleme (quantization), doküman sıkıştırmada önce verinin istatistiği çıkarılır.
  - Sabit uzunluklu kod bütün sembollere aynı bit sayısını verir. Değişken uzunluklu kod sembollere sıklıklarına göre farklı uzunlukta kod verir.
  - Non-adaptif yöntem yalnız belli tipte veride çalışır. Adaptif yöntem her tür veride çalışır, parametreleri değişebilir.
  - Kayıplı yöntemde çözülen veri orijinale dönmez; görüntü ve ses sıkıştırmada kullanılır, çünkü göz ve kulağın algılayamadığı bilgi atılabilir. Kayıpsız yöntem metinlerde tercih edilir; tek bir karakterin kaybı bile anlamı değiştirebilir.
  - Simetrik yöntemde açma, sıkıştırma algoritmasının tersten çalıştırılmasıdır. Asimetrik yöntemde iki taraf farklı algoritmadır, hızları da farklı olabilir.
  - Blok modda her bloğa farklı algoritma uygulanabilir; hangi bloğun hangi algoritmayla kodlandığı başlık (header) bilgisine yazılır.
  - Derlem (corpus): algoritmaların karşılaştırıldığı ortak veri seti (baseline). Başarı birimi metinde bpc (bit per character), görüntüde bpp (bit per pixel).
  - Sıkıştırma oranı = çıktı boyu / girdi boyu. 100 birim 30 birime inerse oran $0{,}3$, başarı $1 - 0{,}3 = \%70$.
  - Görüntüde başarı, açılan görüntünün orijinale benzerliğiyle ölçülür: piksel farklarının karesi alınır, ortalaması ve karekökü alınır (root mean square error). SNR ile e_{RMS} ters yönde değişir.
- Hata bulma (sezme) ve düzeltme
  - Veri iletilirken bazı bitlerde bozulma olabilir; bunlara hata denir (ekrandan, 00:24:30). Atmosferik olaylar (yıldırım vb.) telli ve telsiz ortamlarda hataya yol açar.
  - Bazı bozulmalar kabul edilebilir (faks sayfasının boş beyaz zemininde birkaç pikselin kararması). Bilgi taşıyan kısımdaki bozulma kabul edilemez.
  - Sıkıştırılmış veride tek bitlik bozulma, ardından gelen birçok kodun yanlış çözülmesine yol açabilir.
  - Hatayı sezmek, düzeltmekten çok daha ucuzdur. Bir bitlik hatalar sezilip düzeltilebilir; birden fazla bitlik hatalar çoğu zaman sezilse bile düzeltilemez, verinin yeniden gönderilmesi istenir.
- Sıkıştırma ile güvenlik arasındaki ikilem
  - Sıkıştırma veriyi küçültmeye çalışır; hata sezme ise veriye ek bitler ekler. Diskte sıkıştırmada sorun yoktur, iletim hattı üzerinden gönderilen veride güvenlik de düşünülmelidir. Bu konu aslında sinyal işleme ve haberleşme derslerinin konusudur.
- Hata türleri (ekrandan, 00:30:00)
  - Patlama hatası (burst error): çevre koşulları (yıldırım vb., 1-100 ms) nedeniyle alıcıya anlamsız bilgi gelir.
  - Rastgele hata (random error): iletim yolundaki elektriksel gürültü nedeniyle bilgi bloğunda rastgele bir bit bozulur.
  - Çok kısa süren girişim spike olarak adlandırılır, tek bitlik hataya yol açar. Uzun süren girişim burst hatalara (birden fazla bit) yol açar. Sinyal açıkça 0 ya da 1 olmayıp belirsiz bir alana düşerse buna erasure (silinmiş yer) denir.

- Hata denetim yöntemleri (ekrandan, 00:31:40)
  - FEC (Forward Error Correction, ileri yönlü hata denetimi): veriye ek bitler eklenir; alıcı hatayı bulur ve gerekirse kendisi düzeltir. Hız kaybı yoktur; çok gürültülü ortamlarda kullanılamaz. Yeniden iletimin zor ya da imkânsız olduğu bağlantılarda ve küçük veri kümelerinde kullanılır. Örnekler: katlamalı (convolutional) kodlar, BCH kodları, Hamming kodları, Reed-Solomon kodları.
  - ARQ (Automatic Repeat Request, otomatik tekrar isteği): alıcı hatayı bulur ve bozulan verinin yeniden gönderilmesini ister. Hatayı sezmek için gereken bit sayısı, düzeltmek için gerekenden çok daha az olduğu için uygulamalarda bu teknik yaygındır. Örnekler: CRC kodları, seri eşlik (parity), blok eşlik, modül toplamı.
- Tekrarlama ile hata sezme (ekrandan, 00:40:00)
  - Gönderilen `10110100110` dizisinin dördüncü biti 1 iken 0 alınırsa alıcı hatayı sezemez.
  - Her bit iki kez yazılırsa (1 bit → 2 bit) `11` yerine `01` alındığında hata sezilir ama düzeltilemez; orijinalin `00` mı `11` mi olduğu bilinemez.
  - Her bit üç kez yazılırsa (1 bit → 3 bit) `111` yerine `011` alındığında tek bitlik hata hem sezilir hem çoğunluğa bakılarak düzeltilir. Birden fazla bit bozulursa düzeltilemez.
- Her 2 bit için 5 bitlik kod kelimesi (ekrandan, 00:50:00)
  - Kodlar: `00 → 00001`, `01 → 01010`, `10 → 10100`, `11 → 11111`. Kod kelimeleri birbirinden en az 3 bitte farklı olacak şekilde seçilmiştir.
  - Her kod kelimesinin 5 olası tek bitlik bozulmuş hâli tabloda listelenir; bu 20 bozulmuş kelimenin hiçbiri başka bir kodun bozulmuş hâliyle çakışmaz. Bu yüzden tek bitlik bozulma kesinlikle düzeltilebilir.
  - Örnek: `10 11 00 11` mesajı `10100 11111 00001 11111` olarak gönderilir. `00001` yerine `10001` alınırsa tabloda bu kelime yalnız `00001`'in bozulmuş hâlleri arasında vardır; alıcı onu `00` olarak çözer.
  - İki bit bozulursa (örneğin `11001`) tabloda karşılığı yoktur, düzeltilemez.
- Hamming kodlama (ekrandan, 00:54:50)
  - 4 bitlik veriden 7 bitlik kod kelimesi elde edilen doğrusal bir kodlamadır; bir veri biti yaklaşık $7/4 = 1{,}75$ bitle gönderilir. Bütün kod kelimeleri en az 3 bitte farklıdır; tek bitlik bozulmalar düzeltilebilir.
  - İlk 4 bit orijinal veridir. Son 3 bit eşlik toplamıyla (XOR, $\oplus$) hesaplanır: $b_5 = b_1 \oplus b_2 \oplus b_3$, $b_6 = b_1 \oplus b_3 \oplus b_4$, $b_7 = b_2 \oplus b_3 \oplus b_4$.
  - Örnekler: `1010 → 1010001`, `0111 → 0111001`, `0101 → 0101110`.
  - Alıcıda üç eşlik kontrolü yapılır: $P_1 = \tilde{b}_1 \oplus \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_5$, $P_2 = \tilde{b}_1 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_6$, $P_3 = \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_7$ (ekrandan, 01:00:00 ve 01:06:40; P_3 için aşağıdaki düzeltmeye bakın).
  - $P_1 = P_2 = P_3 = 0$ ise kod kelimesi doğrudur; son üç bit atılır.
  - Bir P_j 1, diğer ikisi 0 ise yalnız o P_j'de bulunan bitin tersi alınır. Örnek: $P_2 = 1$, $P_1 = P_3 = 0$ ise b_6 düzeltilir.
  - Bir P_j 0, diğer ikisi 1 ise o iki kontrolde olup P_j'de olmayan bitin tersi alınır. Örnek: $P_3 = 0$, $P_1 = P_2 = 1$ ise b_1 düzeltilir.
  - Üçü de 1 ise üçünde de bulunan b_3 bozulmuştur.
- Eşlik sınaması (parity) (ekrandan, 01:40:00)
  - Gönderilen veride tek sayıdaki (1, 3, 5 … bit) hatayı sezmek için kullanılır. Eşlik biti 1 ya da 0 yapılarak verideki birlerin sayısı tek ya da çift olarak düzenlenir.
  - Tek eşlik: verideki 1'lerin sayısı tek ise eşlik biti 0 olur. Çift eşlik: verideki 1'lerin sayısı tek ise eşlik biti 1 olur.
  - Örnek veri `0100010100` üç tane 1 içerir; tek eşlikte `0100010100 0`, çift eşlikte `0100010100 1` gönderilir.
  - Çift sayıda bit bozulursa hata sezilmez. Hata sezilse de düzeltilemez; ARQ yöntemidir.
- CRC (Cyclic Redundancy Check) (ekrandan, 01:46:50)
  - Bir üreteç fonksiyonu (polinom) $G(x)$ kullanılır; derecesi CRC kodunun bit uzunluğunu verir. Veri büyüdükçe daha yüksek dereceli bir üreteç seçilir. İki taraf aynı üreteci kullanmalıdır.
  - Örnek: $G(x) = x^3 + x^2 + 1$, katsayıları `1101`; derece 3 olduğu için CRC kodu 3 bittir ve başlangıçta `000`'dır. Mesaj `0111101`, bölünen `0111101000`.
  - XOR'dan sonra en soldaki bit atılır; sıradaki en sol bit 1 ise üreteç (`1101`), 0 ise `0000` yazılıp yeniden XOR yapılır. Slayttaki işlem sonunda CRC `111` bulundu, gönderilen

mesaj `0111101111`.
  - Alıcı aynı işlemi gelen mesaja uygular; sonuç sıfırsa bozulma yoktur (ekrandan, 01:50:00). Üçüncü biti bozuk gelen `0101101111` için sonuç `0001` çıkar, bozulma vardır ve veri yeniden istenir (ekrandan, 01:51:00).
- MD5 ve SHA (ekrandan, 02:00:00)
  - MD5 (Message Digest) bir internet standardıdır; veri güvenliğinde ve bütünlüğünde kullanılır. Her boyuttaki veriden 128 bitlik bir özet (hash) üretir ve bunu 32 bitlik 4 eşit parçaya böler. Veri 512 bitlik bloklara ayrılır, 512'nin katı değilse dolgu (padding) yapılır.
  - Şifrelerin özet olarak saklanması: iyi sistemler şifreyi açık metin olarak saklamaz; girilen şifrenin özeti veritabanındakiyle karşılaştırılır. Şifre unutulunca sistem şifreyi gönderemez, yenisini oluşturmayı ister.
  - MD5'te farklı girdiler için aynı özet çok seyrek de olsa çıkabilir (çakışma). Bu yüzden SHA (Secure Hash Algorithm) önerilmiştir; SHA-1, en çok $2^{64}-1$ bitlik mesajdan 160 bitlik özet üretir.
- Günlük hayattan kontrol rakamlı kodlar
  - UPC (Universal Product Code) bar kodu (ekrandan, 02:03:30): ince-kalın siyah çizgiler ve boşluklar; her ondalık rakam 7 bitle kodlanır. Sol taraftaki 7'li gruplar 0 ile başlar 1 ile biter ve tek sayıda 1 içerir; sağ taraftakiler 1 ile başlar 0 ile biter ve çift sayıda 1 içerir. 12 rakamın dağılımı: 1 rakam ürün tipi + 5 rakam üretici kodu + 5 rakam ürün kodu + 1 kontrol rakamı.
  - UPC kontrol rakamı: $3a_1+a_2+3a_3+a_4+\cdots+a_{10}+3a_{11} \equiv A \pmod{10}$, $C = 10-A$. `04850000139` için toplam 66, $A = 6$, kontrol rakamı 4 (ekrandan, 02:07:40).
  - ABD posta sistemindeki POSTNET bar kodu (ekrandan, 02:10:00): her rakam 5 çizgiyle (kısa = 0, uzun = 1; 3 kısa, 2 uzun) kodlanır. Kontrol rakamı dahil bütün rakamların toplamı 10'a bölünebilmelidir; böylece okunamayan tek bir rakam bulunabilir.
- Sezgisel sıkıştırma (intuitive compression) (ekrandan, 02:20:40)
  - Braille kodlaması: 6 noktalı hücre, $2^6 = 64$ farklı desen. 26 harf ve 10 rakamdan sonra kalan desenler sık kullanılan kelimelere ve harf gruplarına verilir (İngilizcede "and", "for", "of", "the", "with", "ch", "gh", "sh", "th"). Büyük/küçük harf için ayrı kod yerine bir "büyük harf" işareti kullanılır. Türleri: Literary, Nemeth (matematik ve fen), CBC (Computer Braille Code), Music.
  - Mors kodu: nokta 1 birim, çizgi 3 birim; nokta ve çizgiler arası 1 birim, karakterler arası 3 birim, kelimeler arası 6 birim (slayttaki değer; düzeltme aşağıda).
  - Kısaltma ve akronimler de birer sıkıştırmadır: "Yıldız Teknik Üniversitesi" yerine "YTÜ". Akronimde her harf bir kelimeye karşılık gelir; "Dr." gibi kısaltma akronim değildir.
- Sparse string sıkıştırma (ekrandan, 02:25:50)
  - Çok boşluk içeren metin için: karakterlere 0, boşluklara 1 verilir, bitler 4'lü gruplara ayrılır (L1). L2 her L1 grubu için bir bittir (grup sıfırdan farklıysa 1); L3 de L2 için aynısıdır. L1 ve L2'deki sıfır gruplar silinir; L1, L2, L3 ve ardından boşluksuz metin gönderilir.
  - Konu ders sonunda yarım kaldı, sonraki derste devam edilecek.

Hocanın Özellikle Vurguladığı Kısımlar

- Sıkıştırma mı, güvenlik mi?
  - Hata sezmek için eklenen her bit sıkıştırmanın kazancını azaltır ($1 \to 2$, $1 \to 3$, $2 \to 5$ bit). Veri bir yerden bir yere aktarılıyorsa güvenliğe de dikkat edilmeli, ikisinin arası bulunmalıdır. Veri yalnız diskte sıkıştırılıyorsa bu sorun yoktur.
- Hatayı sezmek, düzeltmek demek değildir
  - Sezmek ucuz, düzeltmek pahalıdır. Bu yüzden uygulamalarda çoğu zaman yalnız sezilip veri yeniden istenir (ARQ).
- Sıkıştırılmış veride tek bit bile önemlidir
  - Bir bitin değişmesi, sonraki birçok sembolün farklı kodlarla açılmasına yol açar; örneğin bir harfin kodundaki tek bit değişince başka bir harf çözülür.
- Kod kelimeleri arasındaki en az 3 bitlik fark
  - Tek bitlik bozulmuş hâllerin çakışmaması bu farka dayanır; Hamming kodu da aynı özelliği taşır ve $2 \to 5$'ten daha az ek bitle ($4 \to 7$) çalışır.

- CRC'de soldaki bitin atılması
  - Hoca, bazı kaynakların CRC'yi böyle anlatmadığını, XOR sonrası en soldaki bitin atılıp sıradaki bite göre üreteç ya da sıfır yazılması gerektiğini vurguladı.
- Üreteç fonksiyonu için formül yoktur
  - Derecesi verinin büyüklüğüne göre birkaç denemeyle seçilir; iki taraf aynı üreteci kullanmak zorundadır. Algoritma seçiminde de durum aynıdır: veri tanındıkça hangi yöntemin iyi sonuç verdiği anlaşılır.
- Sıkıştırma yalnız algoritma değildir
  - Braille, Mors, kısaltma ve akronimler de günlük hayatta yapılan sıkıştırmalardır.

Kısa Tekrar Notları

- Hata: iletimde bozulan bit. Burst (çevre koşulları, birden çok bit) ve random (elektriksel gürültü, rastgele bit).
- Spike: çok kısa girişim, tek bit. Erasure: 0 mı 1 mi belirsiz kalan yer.
- FEC: alıcı kendisi düzeltir (Hamming, BCH, Reed-Solomon, katlamalı kodlar). ARQ: alıcı yeniden ister (CRC, eşlik, blok eşlik, modül toplamı).
- 1 → 2 bit: sezer, düzeltemez. 1 → 3 bit: tek bitlik hatayı düzeltir.
- 2 → 5 bit kodları: `00001`, `01010`, `10100`, `11111`; aralarında en az 3 bit fark.
- Hamming (7,4): $b_5 = b_1 \oplus b_2 \oplus b_3$, $b_6 = b_1 \oplus b_3 \oplus b_4$, $b_7 = b_2 \oplus b_3 \oplus b_4$.
- Kontrol: P_1: 1,2,3,5; P_2: 1,3,4,6; P_3: 2,3,4,7. Yalnız $P_1 \to b_5$, yalnız $P_2 \to b_6$, yalnız $P_3 \to b_7$; $P_1, P_2 \to b_1$; $P_1, P_3 \to b_2$; $P_2, P_3 \to b_4$; üçü → b_3.
- Eşlik: tek sayıda hatayı sezer, düzeltmez. Tek eşlikte 1 sayısı tekse eşlik biti 0; çift eşlikte 1.
- CRC: $G(x)$ derecesi = CRC bit sayısı. Alıcıda kalan sıfır değilse bozulma var.
- MD5: 128 bit özet, 512 bitlik bloklar. SHA-1: 160 bit özet.
- UPC: 12 rakam, rakam başına 7 bit; kontrol $C = 10 - (\text{ağırlıklı toplam mod } 10)$.
- POSTNET: rakam başına 5 çizgi (2 uzun); toplam 10'un katı.
- Braille: 6 nokta, 64 desen. Mors: nokta 1, çizgi 3 birim.
- Sparse string: karakter 0, boşluk 1; L1, L2, L3 seviyeleri, sıfır gruplar silinir.

Detaylı Açıklamalar

- **Tek bitlik hatanın etkisi.** Sıkıştırılmış veri, bakınca anlaşılmayan bir 1-0 dizisidir. Bu dizide bir bit değişirse, çözücü o noktadan sonra kod sınırlarını yanlış bulabilir ve birçok sembolü yanlış açar. Hoca bir kelimede "u" harfinin kodundaki tek bit değişince yerine "a"nın kodunun çözüldüğü örneği verdi; okuyan insan bunu tahmin edebilir ama alıcı program bozulmayı anlayamaz. Bu yüzden önce bozulmanın sezilmesi gerekir; bunu veriye eklenen ek bitler sağlar.
- **Tekrarlama kodlarından Hamming'e.** Hiç ek bit yoksa hata sezilemez. Her bit iki kez gönderilince `01` gibi uyumsuz bir çift hatayı gösterir ama orijinalin `00` mı `11` mi olduğu bilinemez. Üç kopyada tek bitlik hatada çoğunluk orijinali verir. Bunun bedeli büyüktür: 1 bitlik veri 2 ya da 3 bitle gönderilir. 2 → 5 bitlik kodda dört kod kelimesi birbirinden en az 3 bitte farklıdır; her kelimenin tek bitlik bozulmuş beş hâli başka hiçbir kelimenin bozulmuş hâliyle çakışmadığı için alıcı tabloya bakarak düzeltebilir. Hamming kodu aynı işi 4 veri bitine 3 eşlik biti ekleyerek yapar; ek bit oranı daha düşüktür.
  - Ders sonrası not: Hoca 2 → 5 kodunda $2/5$ oranını "%40 fazladan bilgi" diye, 1 → 2'yi bir yerde "%100", bir yerde "%50" diye andı. $2/5 = \%40$, kod kelimesinde veri bitlerinin payıdır; eklenen bit sayısı veriye göre $3/2 = \%150$, kod kelimesine göre $3/5 = \%60$'tır. 1 → 2'de veri payı $\%50$, eklenen bit veriye göre $\%100$'dür. Hamming (7,4) için veri payı $4/7 \approx \%57$'dir.
- **Hamming çözme örneği.** Hocanın örneğinde (ekrandan, 01:06:40) `0101` verisi `0101110` olarak kodlanır; birinci bit bozulur ve `1101110` alınır. $P_1 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$, $P_2 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$, $P_3 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$. P_1 ve P_2'de olup P_3'te olmayan bit b_1'dir; tersi alınınca `0101110` ve veri `0101` elde edilir.
  - Ders sonrası not: Slayttaki kontrol denkleminde $P_3 = \tilde{b}_2 \oplus \tilde{b}_3 \oplus \tilde{b}_4 \oplus \tilde{b}_5$ yazıyordu. Hoca derste bunun hatalı olduğunu fark etti ve doğrusunun $\tilde{b}_7$ olduğunu söyleyip düzeltti (01:05). Doğru denklem b_7'nin kodlama formülüyle uyumludur. Hocanın düzeltmeden önce "b_5 hem P_1'de hem P_3'te" dediği yer bu hatadan kaynaklanır; doğru

denklemlerle b_5 yalnız P_1'dedir.

- **CRC hesabı.** Bölünen, mesajın sonuna üretecin derecesi kadar sıfır eklenerek elde edilir (`0111101 + 000`). Slayttaki el yazısı işlemde ilk adımda baştaki bit 0 olduğu hâlde `0111` ile `1101` XOR'lanmış, sonra her adımda en soldaki bit atılıp sıradaki bite göre `1101` ya da `0000` yazılmıştır. Bu işlemle CRC `111` çıkar ve alıcı aynı işlemle `0111101111` için `000` bulur.
  - Ders sonrası not: Standart CRC'de (modulo-2 polinom bölmesi) baştaki bit 0 iken üreteç değil `0000` ile XOR yapılır. Bu kuralla `0111101000` mod `1101` kalanı `011`'dir (python ile hesaplandı); gönderilen mesaj `0111101011` olur ve alıcıda kalan `000` çıkar. Slayttaki `111`, ilk adımda üretecin 0 ile başlayan gruba uygulanmasından kaynaklanır. Gönderen ve alan aynı işlemi yaptığı için slayttaki örnekte de bozulma sezilmektedir.
- **MD5 hakkında.** Hoca MD5'in özet ürettiğini ve şifrelerin bu özetle saklandığını, şifre unutulunca sistemin şifreyi geri veremediğini anlattı.
  - Ders sonrası not: Slaytta ve anlatımda bir yerde "özet bilgiden orijinal veriye dönülür" ifadesi geçti; aynı slaytta MD5'in tek yönlü olduğu, geri dönüşümü olmadığı da yazıyor. Doğrusu ikincisidir: özet fonksiyonundan orijinal veri geri elde edilmez, şifre kontrolü özetlerin karşılaştırılmasıyla yapılır.
  - Ders sonrası not: Hoca MD5'in kırılabildiğinin ispat edilmediğini ve MD5'in SHA'dan daha kuvvetli olduğunu söyledi. MD5'te pratik çakışma üretme yöntemleri büyük olasılıkla 2004'ten beri bilinmektedir; MD5 bugün güvenlik amaçlı (şifre saklama, imza) önerilmez, SHA-2 ailesi gibi daha güçlü algoritmalar kullanılır. MD5'in dolgusu da yalnız sıfırlardan oluşmaz; önce bir 1 biti, sonra sıfırlar ve en sonda mesaj uzunluğu eklenir.
- **Mors kodunda kelime arası.** Slaytta kelimeler arası 6 birim yazıyor.
  - Ders sonrası not: Uluslararası Mors kodunda kelimeler arası boşluk büyük olasılıkla 7 birimdir. Hocanın "Alfred Fail" diye andığı kişi Alfred Vail'dir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.